Technical Guide: Implementing AspeQt-2k26 on Raspberry Pi 4/5

Subtitle: High-Speed SIO Emulation via Native GPIO and Qt6

Author: 13leader.net

Date: January 2026

Project Repo: https://github.com/pjones1063/AspeQt-2k26

---

1. Overview

As modern operating systems deprecate the Qt5 libraries, maintaining stable SIO (Serial Input/Output) connectivity for the Atari 8-bit family requires a transition to Qt6. **AspeQt-2k26** provides this future-proofing while introducing native TNFS browsing. This document outlines the steps to install the portable 64-bit build on a Raspberry Pi and interface it directly with Atari hardware via the GPIO header.

2. Installation (ARM64 Portable)

Instead of utilizing standard repository installs which may carry legacy dependencies, use the optimized portable tarball:

1. **Download:**

   ```
   $ sudo apt update

   $ sudo apt install qt6-base-dev qt6-base-private-dev
   libqt6gui6 libqt6widgets6 libqt6network6

   $ wget https://github.com/pjones1063/AspeQt-2k26/release
   (add current release - Pi tarball)
   ```

2. **Extract:**

   ```
   $ tar -xzvf AspeQt-2K26-pi.64-portable.tar.gz
   ```

3. **Execute:**

```
$ cd AspeQt-2K26-pi.64-portable

$ ./AspeQt
```

3. Hardware Interfacing (GPIO to SIO)

The Raspberry Pi 4/5 can communicate directly with the Atari SIO bus using its internal UART.

[CRITICAL SAFETY NOTE]

Atari SIO logic levels are 5V. The Raspberry Pi GPIO header operates at **3.3V**. Connecting these directly will result in permanent damage to the Pi's SoC. A **Logic Level Shifter** is mandatory for this project.

Wiring Diagram Reference

Connection Logic

Atari SIO Pin	Signal Description	Raspberry Pi GPIO (Physical Pin)
Pin 3	Data Out (Atari TX)	**Pin 10 (GPIO 15 / RXD)**
Pin 5	Data In (Atari RX)	**Pin 8 (GPIO 14 / TXD)**
Pin 4	Ground (GND)	**Pin 6 (Ground)**

4. System Configuration

Before AspeQt can access the serial hardware, the Pi’s default "Serial Console" must be disabled to release `/dev/serial0`.

1. Open the configuration tool: `sudo raspi-config`
2. Navigate to **Interface Options** -> **Serial Port**.
3. Set **Login shell over serial** to **NO**.
4. Set **Hardware serial port** to **YES**.
5. **Finish** and **Reboot**.

5. Software Configuration

Launch **AspeQt-2k26** and enter the **Options > Configure** menu:

- **Serial Port:** Select /dev/serial0.
- **Handshake:** Set to **None** (unless you have wired the PROCEED/INTERRUPT lines).
- **Baud Rate:** Standard start is **19,200**. Once verified, the Qt6 engine and Pi 4/5 hardware comfortably support high-speed divisors (up to 127kbps+).

6. Troubleshooting & Performance Tuning

Interfacing vintage 1980s hardware with a modern 64-bit Linux environment can sometimes present timing hurdles. If you encounter "Timeout" errors or "Checksum" failures, check the following:

Common SIO Timing Issues

- **The "Slow-Speed" Fallback:** If your Atari fails to boot the first time, ensure the baud rate is locked at **19,200**. Some OS loaders require the initial handshake to be strictly at the standard speed before switching to high-speed routines.

- **Linux Latency (CPU Scaling):** By default, the Pi may throttle its CPU to save power, which can jitter the bit-banging timing of the UART.
  - *Fix:* Set your Pi to "Performance" mode by running: `echo performance | sudo tee /sys/devices/system/cpu/cpu*/cpufreq/scaling_governor`

- **The Level Shifter Bottleneck:** Cheap bi-directional level shifters sometimes struggle with high-frequency signals. If you cannot reach speeds above 54kbps, ensure your shifter is rated for high-speed I2C/SPI (at least 2MHz).

Handshaking and GPIO 14/15

- **Permissions:** If AspeQt cannot open `/dev/serial0`, ensure your user is part of the dialout group: `sudo usermod -a -G dialout $USER` (requires a log-out/log-in to take effect).

Document provided by 13leader.net *Visit the BBS via Telnet for community support and TNFS updates.*